

Bản sơ về các thuật toán luồng mạng dựa trên tư tưởng phân tầng

Wang Xinshang
Trường Trung học Diên An, Thượng Hải

2007

Nguồn gốc: Bản sơ về các thuật toán luồng mạng dựa trên tư tưởng phân tầng

IOI China candidate team papers / 2007 / day 1 / Wang Xinshang

Abstract

Bài viết giới thiệu tương đối chi tiết ba thuật toán luồng mạng dựa trên khái niệm đồ thị phân tầng, đồng thời dùng các bài toán mẫu để minh họa ưu thế của thuật toán Dinic trong các kỳ thi tin học.

Từ khóa. Đồ thị phân tầng; luồng mạng; thuật toán cơ bản; ứng dụng; MPLA; Dinic; MPM.

Contents

1	Dẫn nhập	2
2	Các khái niệm chuẩn bị	2
2.1	Đồ thị dư	2
2.2	Mức của đỉnh	3
2.3	Đồ thị phân tầng	3
2.4	Luồng chặn	3
3	Thuật toán tăng theo đường ngắn nhất MPLA	3
3.1	Các bước thuật toán	4
3.2	Số pha	4
3.3	Phân tích độ phức tạp	5
4	Thuật toán Dinic	5
4.1	Các bước thuật toán	5
4.2	Minh họa bằng lời	6
4.3	Phân tích độ phức tạp	6
5	Ứng dụng của Dinic trong thi tin học	7
5.1	Bài toán 1: Lợi nhuận lớn nhất	7
5.2	Bài toán 2: Trò chơi ma trận	8

6 Thuật toán MPM	9
6.1 Các bước thuật toán	9
6.2 Phân tích độ phức tạp	10
7 Tổng kết	10

1 Dẫn nhập

Lý thuyết đồ thị là một ngành vừa cổ điển vừa trẻ trung, và chiếm tỉ trọng rất lớn trong các kỳ thi tin học.¹ Trong đó, các thuật toán luồng mạng xuất hiện khá thường xuyên. Kiến thức về luồng mạng rất phong phú: từ định lý luồng cực đại–cắt cực tiểu cơ bản, các biến thể của mô hình mạng, cho tới nhiều kỹ thuật tối ưu cho thuật toán đẩy luồng trước. Muốn ứng phó với các dạng bài khác nhau, học sinh cần tích lũy dần những kiến thức này.

Khi các kỳ thi phát triển, độ khó và phạm vi kiểm tra cũng tăng lên. Với nhiều bài toán mới, chỉ nắm thuật toán tăng luồng thô sơ nhất là chưa đủ. Bài viết này trình bày ba thuật toán luồng mạng dựa trên tư tưởng phân tầng, hướng tới các bài toán có dữ liệu lớn hơn, và thông qua so sánh để làm rõ cách dùng chúng khi giải bài. Những kiến thức nền như định lý luồng cực đại–cắt cực tiểu sẽ không được chứng minh lại ở đây; độc giả có thể tìm thấy trong nhiều tài liệu khác về luồng mạng.

2 Các khái niệm chuẩn bị

Những định lý nền tảng như định lý luồng cực đại–cắt cực tiểu không được chứng minh lại ở đây; bài viết chỉ nhắc các khái niệm cần cho ba thuật toán phân tầng.²

2.1 Đồ thị dư

Cho một mạng luồng $G_1 = (V_1, E_1)$, nguồn s , đích t , hàm sức chứa c , và một hàm luồng khả thi f trên mạng đó. Đồ thị dư tương ứng $G_2 = (V_2, E_2)$ được định nghĩa như sau. Tập đỉnh không đổi:

$$V_2 = V_1.$$

Với mỗi cạnh có hướng $(u, v) \in E_1$:

- nếu $f(u, v) < c(u, v)$, đưa cạnh (u, v) vào E_2 với trọng số

$$g(u, v) = c(u, v) - f(u, v);$$

- nếu $f(u, v) > 0$, đưa cạnh ngược (v, u) vào E_2 với trọng số

$$g(v, u) = f(u, v).$$

Vì vậy, mỗi cạnh của mạng ban đầu biến thành một hoặc hai cạnh trong đồ thị dư. Mỗi cạnh dư cho biết ta còn có thể tăng luồng theo hướng ấy trong mạng gốc; trọng số $g(a, b)$ chính là lượng luồng lớn nhất có thể tăng theo hướng từ a tới b ở thời điểm hiện tại. Do đó, mọi đường đi đơn từ s tới t trong đồ thị dư tương ứng với một đường tăng luồng, và giá trị nhỏ nhất của các trọng số trên đường đi là lượng luồng lớn nhất có thể tăng trong một lần theo đường ấy.

¹Nguồn gốc của lý thuyết đồ thị thường được gắn với lời giải bài toán bảy cây cầu của Euler vào thế kỷ XVIII, nhưng sự phát triển hiện đại mạnh mẽ của ngành này chủ yếu bắt đầu từ thập niên 1950–1960. Vì vậy bản gốc gọi đồ thị là một ngành vừa cổ xưa vừa trẻ.

²Trong bài này, nếu không nói khác, mọi cạnh đều là cạnh có hướng. Đường đi đơn là đường đi không lặp đỉnh hoặc cạnh.

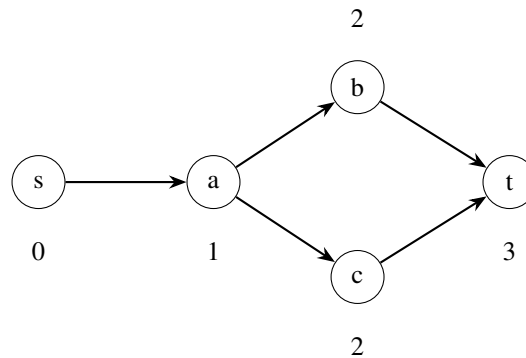
2.2 Mức của đỉnh

Trong đồ thị dư, độ dài đường đi ngắn nhất từ nguồn s tới đỉnh u được gọi là **mức** của u , ký hiệu $\text{level}(u)$. Nguồn có mức 0. Nếu một đỉnh không đi tới được từ nguồn trong đồ thị dư, mức của nó không được xác định trong pha hiện tại.

Một cách nhìn đơn giản là chia các đỉnh thành từng lớp:

$$L_i = \{u \mid \text{level}(u) = i\}, \quad i = 0, 1, 2, \dots$$

Mỗi lần chạy BFS từ nguồn trên đồ thị dư, ta thu được các lớp này.



Một đồ thị dư minh họa: số bên cạnh đỉnh là mức của đỉnh đó.

2.3 Đồ thị phân tầng

Đồ thị phân tầng $G_3 = (V_3, E_3)$ được dựng trên đồ thị dư $G_2 = (V_2, E_2)$. Với một cạnh dư (u, v) , ta giữ cạnh ấy trong E_3 khi và chỉ khi

$$\text{level}(u) + 1 = \text{level}(v).$$

Tập đỉnh V_3 gồm các đỉnh kề với ít nhất một cạnh trong E_3 , cùng với nguồn và đích khi cần xét đường tăng luồng.

Trực giác là: đồ thị phân tầng là “đồ thị các đường ngắn nhất” xây trên đồ thị dư. Nếu bắt đầu từ nguồn và đi theo các cạnh trong đồ thị phân tầng, thì sau k cạnh ta luôn đứng ở một đỉnh có khoảng cách ngắn nhất bằng k so với nguồn trong đồ thị dư.

2.4 Luồng chặn

Xét một luồng khả thi f trong mạng. Nếu trong đồ thị phân tầng của đồ thị dư tương ứng không còn đường tăng luồng từ s tới t , ta gọi f là **luồng chặn** của đồ thị phân tầng ấy.

Nói cách khác, luồng chặn không nhất thiết là luồng cực đại của toàn mạng, mà chỉ chặn hết mọi đường tăng luồng ngắn nhất trong pha hiện tại.

3 Thuật toán tăng theo đường ngắn nhất MPLA

MPLA là viết tắt của *maximum path length augmenting* trong bản dịch này: ta luôn làm việc trên các đường tăng luồng ngắn nhất bằng cách dựng đồ thị phân tầng. Nội dung chính của thuật toán là lặp lại việc dựng tầng và tăng luồng trong đồ thị tăng cho tới khi không còn đường tăng luồng.

3.1 Các bước thuật toán

1. Khởi tạo luồng, từ đó có đồ thị dư ban đầu.
2. Chạy BFS trên đồ thị dư để tính mức các đỉnh, tức dựng đồ thị phân tầng. Nếu đích không xuất hiện trong các mức, thuật toán kết thúc.
3. Trong đồ thị phân tầng, liên tục dùng BFS tìm đường tăng luồng và tăng luồng theo đường đó cho tới khi đồ thị phân tầng không còn đường tăng luồng.
4. Quay lại bước 2.

Một lần thực hiện các bước 2 và 3 được gọi là một **pha**. Trong mỗi pha, ta dựng đồ thị phân tầng từ đồ thị dư hiện tại, sau đó tìm một luồng chặn trong đồ thị phân tầng ấy. Sau khi không còn đường tăng luồng trong tầng, thuật toán sang pha tiếp theo.

Khi cài đặt, không cần thật sự tạo một cấu trúc đồ thị mới cho G_3 . Chỉ cần gán level cho từng đỉnh, rồi khi duyệt cạnh (u, v) kiểm tra điều kiện $\text{level}(u) + 1 = \text{level}(v)$.

3.2 Số pha

Định lý. Trong một mạng có n đỉnh, thuật toán MPLA có nhiều nhất n pha.

Chứng minh. Giả sử sau khi dựng đồ thị phân tầng, độ dài đường ngắn nhất từ s tới t trong đồ thị dư là k . Chia các đỉnh đi tới được từ nguồn thành $k + 1$ lớp:

$$L_i = \{u \mid \text{level}(u) = i\}, \quad 0 \leq i \leq k.$$

Trong đồ thị dư lúc đó, các cạnh có thể được nhìn theo hai loại quan trọng:

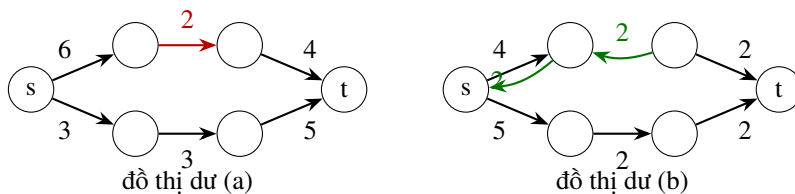
- cạnh loại một đi từ L_i tới L_{i+1} ;
- cạnh loại hai đi từ L_i tới L_j với $j \leq i$.

Đồ thị phân tầng chỉ chứa cạnh loại một. Vì vậy mọi đường tăng luồng trong tầng đều gồm đúng các cạnh loại một và có độ dài k .

Mỗi lần tăng luồng theo một đường trong tầng, ít nhất một cạnh trên đường bị bão hòa và bị xóa khỏi đồ thị dư theo hướng thuận. Đồng thời, các cạnh ngược của đường tăng luồng được thêm vào đồ thị dư; những cạnh ngược này đều đi từ lớp sâu hơn về lớp nông hơn, tức thuộc loại hai.

Khi đã tìm được luồng chặn, không còn đường nào chỉ đi qua các cạnh loại một từ L_0 tới L_k . Mọi đường mới từ s tới t , nếu còn tồn tại trong đồ thị dư, bắt buộc phải dùng ít nhất một cạnh loại hai để lùi lên một lớp trước khi đi tiếp xuống các lớp sau. Vì đã phải lùi lại, số cạnh loại một cần đi sau đó tăng lên, nên tổng độ dài đường đi lớn hơn k .

Do đó, độ dài đường tăng luồng ngắn nhất tăng nghiêm ngặt sau mỗi pha. Độ dài nhỏ nhất có thể là 1, lớn nhất có thể là $n - 1$, cộng thêm lần cuối cùng khi đích không còn đạt được từ nguồn, nên số pha không vượt quá n .



Sau một lần tăng luồng, một cạnh thuận trong tầng có thể bị xóa, còn các cạnh ngược mới thuộc loại đi lùi về lớp trước.

3.3 Phân tích độ phức tạp

Tách chi phí thành hai phần: dựng đồ thị phân tầng và tìm đường tăng luồng trong một tầng.

Mỗi pha dựng tầng bằng một BFS trên đồ thị dư, chi phí $\mathcal{O}(m)$. Vì có nhiều nhất n pha, tổng chi phí dựng tầng là

$$\mathcal{O}(nm).$$

Trong một pha, mỗi lần tăng luồng làm bão hòa ít nhất một cạnh của đồ thị phân tầng. Đồ thị phân tầng có nhiều nhất m cạnh, nên số lần tăng luồng trong một pha không vượt quá m . Với MPLA, mỗi lần lại dùng BFS để tìm đường tăng luồng; chi phí tìm đường là $\mathcal{O}(m + n)$, và chi phí sửa luồng dọc đường là $\mathcal{O}(n)$. Vì vậy chi phí mỗi pha là

$$\mathcal{O}(m(m + n)) = \mathcal{O}(m^2)$$

trong các mạng thường xét với $m \geq n$. Tổng chi phí tăng luồng qua mọi pha là $\mathcal{O}(nm^2)$, và đây cũng là độ phức tạp tổng quát của MPLA.

4 Thuật toán Dinic

4.1 Các bước thuật toán

Dinic cũng chia quá trình thành các pha trên đồ thị phân tầng. Khác biệt then chốt so với MPLA là: trong mỗi pha, Dinic dùng một quá trình DFS để tìm luồng chặn, thay vì nhiều lần BFS độc lập.

1. Khởi tạo luồng và đồ thị dư.
2. Chạy BFS trên đồ thị dư để tính mức các đỉnh. Nếu không tới được đích, thuật toán kết thúc.
3. Dùng DFS trên đồ thị phân tầng để tăng luồng cho tới khi thu được luồng chặn.
4. Quay lại bước 2.

Quá trình DFS trong một pha có thể mô tả bằng một đường hiện tại p , ban đầu chỉ gồm nguồn s . Ký hiệu $p.top$ là đỉnh cuối của đường.

```
p <- [s]
while outdeg(s) > 0 do
  u <- p.top
  if u != t then
    if outdeg(u) > 0 then
      choose a level edge (u, v)
      push v to the end of p
    else
      delete u from p and from the level graph
      delete all level-graph edges incident to u
  else
    augment along p and delete saturated edges on p
    move p.top back to the last reachable vertex on p
end while
```

Nếu đỉnh cuối của p là đích, ta đã có một đường tăng luồng. Sau khi tăng luồng, ít nhất một cạnh trên đường bị bão hòa; đường hiện tại được lùi về đỉnh xa nhất trên p vẫn còn đi tới được từ nguồn theo các cạnh chưa bị xóa.

Nếu đỉnh cuối u không phải đích, có hai khả năng. Nếu còn cạnh tăng đi ra từ u , ta đi tiếp theo cạnh đó. Nếu không còn cạnh đi ra, u không thể nằm trên bất kỳ đường tăng luồng nào của phần đồ thị tăng còn lại; khi đó xóa u , xóa các cạnh kề với u , rồi lùi đường hiện tại tới một đỉnh.

Quá trình trên kết thúc khi mọi cạnh đi ra từ nguồn trong đồ thị tăng đã bị xóa. Khi đó đồ thị tăng không còn đường từ s tới t , tức ta đã tìm được một luồng chặn.

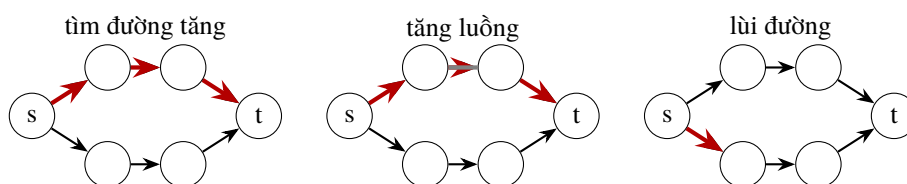
4.2 Minh họa bằng lời

Trong một đồ thị phân tầng, Dinic luôn giữ một đường hiện tại từ nguồn đi dần về phía đích:

$$s \rightarrow \dots \rightarrow u.$$

Khi còn cạnh hợp lệ từ u , đường được kéo dài. Khi gặp đích, ta đẩy thêm luồng trên cả đường; các cạnh bão hòa bị loại khỏi pha hiện tại. Khi gặp một đỉnh cắt, đỉnh đó bị xóa vì không thể đóng góp cho đường tăng luồng nào nữa.

Do chỉ đi theo cạnh từ lớp i sang lớp $i + 1$, DFS không tạo chu trình trong một pha. Việc xóa cạnh và xóa đỉnh là nguyên nhân chính giúp Dinic tránh lặp lại nhiều phép tìm đường tốn kém như MPLA.



Minh họa quá trình DFS trong Dinic: tìm đường hiện tại, tăng luồng làm bão hòa cạnh, rồi lùi về đỉnh còn có thể mở rộng.

4.3 Phân tích độ phức tạp

Lập luận về số pha giống MPLA: sau mỗi pha, độ dài đường tăng luồng ngắn nhất tăng nghiêm ngặt, nên Dinic có nhiều nhất n pha. Tổng chi phí dựng đồ thị phân tầng vẫn là $\mathcal{O}(nm)$.

Xét chi phí DFS trong một pha.

Sửa luồng và lùi sau khi tăng. Trong một pha, số lần tăng luồng không vượt quá m , vì mỗi lần tăng làm bão hòa ít nhất một cạnh tăng. Mỗi lần sửa luồng dọc đường mất $\mathcal{O}(n)$, và việc lùi đường hiện tại cũng mất $\mathcal{O}(n)$. Vì vậy phần này tốn

$$\mathcal{O}(mn)$$

cho một pha.

Đi tới và lùi khi duyệt DFS. Mỗi lần DFS đi tới, nó đi theo một cạnh từ lớp hiện tại sang lớp kế tiếp. Một đường đơn trong tầng có độ dài nhiều nhất n . Khi gặp đỉnh cắt, thuật toán xóa đỉnh đó; số lần lùi kiểu này trong một pha nhiều nhất n .

Trong trường hợp xấu nhất, có n lần đi tới bị xóa bỏ cùng với các đỉnh cắt. Các lần đi tới còn lại đều góp phần vào những đường tăng luồng đã tìm được. Có nhiều nhất m đường tăng luồng trong một pha,

mỗi đường dài nhiều nhất n , nên tổng số bước đi tới là $\mathcal{O}(mn)$. Cộng thêm các lần lùi do đỉnh cắt, chi phí duyệt DFS trong một pha vẫn là $\mathcal{O}(mn)$.

Vì thế, chi phí tìm luồng chặn trong một pha là $\mathcal{O}(mn)$. Nhân với nhiều nhất n pha, độ phức tạp tổng quát của Dinic là

$$\mathcal{O}(mn^2).$$

Trong thực tế thi đấu, Dinic thường nhanh hơn đánh giá tổng quát này rất nhiều, đặc biệt trên các mạng hai phía và các mạng có sức chứa nguyên nhỏ.

5 Ứng dụng của Dinic trong thi tin học

5.1 Bài toán 1: Lợi nhuận lớn nhất

Mô tả bài toán. Đây là bài NOI 2006.³ Có N trạm trung chuyển tín hiệu liên lạc. Xây trạm i tốn chi phí P_i . Ngoài ra có M nhóm người dùng. Nhóm j sẽ dùng hai trạm A_j và B_j để liên lạc; nếu phục vụ nhóm này, công ty thu được C_j . Hãy chọn xây một số trạm sao cho lợi nhuận ròng là lớn nhất.

Giới hạn trong bài gốc: $N \leq 5000$, $M \leq 50000$.

Mô hình hóa ban đầu. Biểu diễn trạm i bằng một đỉnh có trọng số $-P_i$. Biểu diễn nhóm người dùng j bằng một cạnh nối hai đỉnh A_j, B_j , có trọng số C_j . Ta cần chọn một tập đỉnh sao cho tổng trọng số các đỉnh đã chọn cộng với tổng trọng số các cạnh có cả hai đầu mút trong tập là lớn nhất.

Hãy tưởng tượng ban đầu ta định xây mọi trạm và phục vụ mọi nhóm người dùng: ta thu $\sum C_j$ và trả $\sum P_i$. Mỗi cạnh lợi nhuận có thể dùng một phần giá trị của nó để bù chi phí cho hai đầu mút. Nếu một đỉnh vẫn âm sau khi mọi cạnh kề đã góp hết, giữ nó là không có lợi; ta nên bỏ trạm đó và bỏ các nhóm người dùng cần trạm ấy. Việc bỏ một đỉnh có thể khiến các đỉnh khác mất phần bù chi phí và cũng trở nên không có lợi. Quá trình này chính là một bài toán cắt tối thiểu.

Mạng luồng. Dựng mạng hai phía như sau.

- Với mỗi nhóm người dùng j , tạo đỉnh X_j và nối $S \rightarrow X_j$ với sức chứa C_j .
- Với mỗi trạm i , tạo đỉnh Y_i và nối $Y_i \rightarrow T$ với sức chứa P_i .
- Với nhóm j , nối $X_j \rightarrow Y_{A_j}$ và $X_j \rightarrow Y_{B_j}$, mỗi cạnh có sức chứa INF.

Một lát cắt hữu hạn không thể cắt các cạnh INF, do đó nếu giữ lợi nhuận của nhóm j ở phía nguồn thì cũng phải giữ cả hai trạm tương ứng ở phía nguồn. Giá trị lát cắt tối thiểu bằng

chi phí các trạm được giữ + lợi nhuận ban đầu của các nhóm bị bỏ.

Vì tổng lợi nhuận ban đầu $\sum C_j$ là hằng số, đáp án là

$$\sum_{j=1}^M C_j - \text{maxflow}.$$

³Các thời gian chạy trong phần này là kết quả thử nghiệm chương trình của tác giả, độ lệch chuẩn được ghi khoảng $\sigma = 0.02$ giây.

So sánh thuật toán. Với dữ liệu lớn, MPLA thường không qua được các bộ kiểm thử lớn. Tác giả ghi nhận thời gian đại diện như sau:

Thuật toán	Test 1–8	Test 9	Test 10
MPLA	< 0.1s	> 30s	> 30s
Dinic	< 0.03s	0.40s	0.37s
Đẩy luồng trước	< 0.03s	0.53s	0.51s

Một hướng khác là dựng luồng ban đầu bằng tham lam, chẳng hạn ưu tiên các đỉnh bậc nhỏ trong đồ thị hai phía. Tuy nhiên cách này phụ thuộc nhiều vào đặc điểm đồ thị của bài. Dinic ngắn gọn hơn, ít sai hơn, và trong bài này đủ nhanh để không cần dựa vào luồng ban đầu đặc biệt.

Ở đây các phép tăng luồng trước đơn giản, như hai lần BFS đẩy thật nhiều luồng từ nguồn tới đích rồi đẩy phần dư ngược lại, hầu như không cải thiện tốc độ: chương trình chính cũng sẽ đạt cùng trạng thái sau một lượng công tương tự. Vì thế, nếu muốn tham lam luồng ban đầu, nó phải dựa thật sự vào cấu trúc của bài. Nguồn gợi ý của tác giả là báo cáo AHTSC 2006 của Zhou Yuan, nơi kỹ thuật ưu tiên đỉnh bậc nhỏ từng được dùng trong bài toán ghép cặp hai phía.

Nhận xét. Trên mạng hai phía của bài này, Dinic hơi nhanh hơn đẩy luồng trước theo số liệu của tác giả. Điều đó không có nghĩa Dinic luôn nhanh hơn mọi biến thể đẩy luồng trước; mỗi thuật toán có loại đồ thị thuận lợi riêng. Lợi thế lớn của Dinic trong thi đấu là cân bằng tốt giữa tốc độ, độ dễ hiểu và độ dễ cài.

5.2 Bài toán 2: Trò chơi ma trận

Mô tả bài toán. Cho các số $R_1, \dots, R_n$ và $C_1, \dots, C_m$. Hỏi có tồn tại ma trận 0-1 kích thước $n \times m$ sao cho hàng i có đúng R_i ô bằng 1, cột j có đúng C_j ô bằng 1. Ngoài ra, ở mỗi hàng và mỗi cột có nhiều nhất một ô bị chỉ định bắt buộc bằng 0. Giới hạn: $n, m \leq 1000$, mỗi bộ dữ liệu có nhiều nhất 10 test.

Mô hình luồng trực tiếp. Dựng mạng hai phía:

- tạo đỉnh X_i cho hàng i , nối $S \rightarrow X_i$ với sức chứa R_i ;
- tạo đỉnh Y_j cho cột j , nối $Y_j \rightarrow T$ với sức chứa C_j ;
- nếu ô (i, j) không bị bắt buộc bằng 0, nối $X_i \rightarrow Y_j$ với sức chứa 1.

Nếu cạnh $X_i \rightarrow Y_j$ mang một đơn vị luồng, đặt ô $(i, j) = 1$. Bài toán có lời giải khi và chỉ khi luồng cực đại bằng tổng số ô bằng 1, tức bằng $\sum_i R_i = \sum_j C_j$.

Cách dựng này có $\mathcal{O}(nm)$ cạnh. Với MPLA chỉ đạt được một phần điểm; với Dinic có thể đạt điểm cao hơn, nhưng vẫn chưa phải lời giải chuẩn của bài.

Một kiểm tra tham lam hỗ trợ. Tạm bỏ qua các ô bị ép bằng 0. Vì hoán vị toàn bộ các hàng hoặc các cột không làm thay đổi bản chất bài toán, có thể sắp R_i và C_j giảm dần. Khi xét lần lượt các cột, ta tưởng tượng mỗi hàng có thể cung cấp một ô 1 cho các cột đầu tiên còn cần. Nếu một cột cần nhiều ô 1 hơn số hàng đang có thể cung cấp, ta dùng các ô đã “dư” từ những cột trước. Nếu tới một cột mà kể cả dùng hết phần dư vẫn không đủ, thì chắc chắn không tồn tại ma trận.

Kiểm tra này loại được nhiều trường hợp vô nghiệm trước khi chạy luồng. Trong bài gốc, thêm kiểm tra đơn giản ấy rồi dùng Dinic có thể qua toàn bộ dữ liệu, trong khi MPLA vẫn không đủ nhanh.

	6	6	5	4	4
6	1	1	1	1	1
5	1	1	1	1	0
5	1	1	1	1	0
4	1	1	1	0	0
4	1	1	0	0	0
3	1	1	0	0	0
2	0	1	0	0	0

Sơ đồ trên diễn giải dữ liệu ví dụ: các tổng cột nằm phía trên, các tổng hàng nằm bên trái. Khi quét từ trái qua phải, những hàng còn khả năng cung cấp 1 cho cột hiện tại được dùng trước; phần dư của cột trước chỉ có thể bù cho cột sau nếu nó không vi phạm việc mỗi hàng chỉ được đặt một ô 1 trong cột đang xét. Tối một cột mà mọi phần dư hợp lệ vẫn không đủ, ta kết luận vô nghiệm.

Nhận xét. Lời giải chuẩn của bài là một thuật toán tham lam đầy đủ. Dù vậy, ví dụ này cho thấy trong thi đấu, một thuật toán luồng đủ mạnh có thể giúp ta đạt kết quả tốt khi chưa kịp khai thác hết cấu trúc riêng của bài.

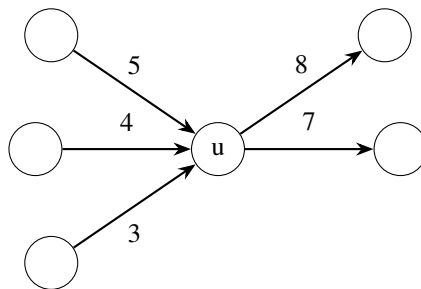
6 Thuật toán MPM

6.1 Các bước thuật toán

MPM cũng tìm luồng chặn theo từng pha trên đồ thị phân tầng, nhưng không dùng DFS kiểu Dinic. Trước hết, định nghĩa **thông lượng** của một đỉnh $u \neq s, t$ trong đồ thị phân tầng:

$$\text{throughput}(u) = \min \left(\sum_{(v,u)} g(v,u), \sum_{(u,w)} g(u,w) \right).$$

Đây là lượng luồng lớn nhất có thể đi xuyên qua u nếu chỉ xét cân bằng giữa tổng sức chứa đi vào và tổng sức chứa đi ra của u .



Ở đỉnh u , tổng sức chứa vào là $4 + 5 + 3 = 12$, tổng sức chứa ra là $7 + 8 = 15$, nên $\text{throughput}(u) = 12$.

Trong mỗi pha, MPM tìm luồng chặn như sau.

1. Tìm một đỉnh u có $\text{throughput}(u)$ nhỏ nhất trong đồ thị phân tầng, không tính nguồn và đích.
2. Từ u , đẩy $\text{throughput}(u)$ đơn vị luồng về phía đích trong đồ thị phân tầng.
3. Từ u , kéo $\text{throughput}(u)$ đơn vị luồng từ phía nguồn trong đồ thị phân tầng.
4. Xóa u . Nếu đồ thị phân tầng chỉ còn nguồn và đích, hoặc không còn đường từ nguồn tới đích, pha kết thúc; nếu không, quay lại bước 1.

Vì u là đỉnh có thông lượng nhỏ nhất, quá trình đẩy và kéo lượng throughput(u) sẽ không bị kẹt ở các đỉnh khác. Khi đẩy hoặc kéo, ta luôn cố gắng làm bão hòa các cạnh đã dùng; tại mỗi bước chỉ có thể còn nhiều nhất một cạnh chưa bão hòa hoàn toàn theo hướng đang xử lý.

6.2 Phân tích độ phức tạp

Trong một pha, mỗi vòng của MPM xóa một đỉnh, nên có nhiều nhất n vòng. Nếu chọn đỉnh có thông lượng nhỏ nhất bằng cách quét tuyến tính, mỗi vòng tốn $\mathcal{O}(n)$. Mỗi lần đẩy và kéo xử lý nhiều nhất $\mathcal{O}(n)$ cạnh chưa bão hòa hoàn toàn, còn trong cả pha tổng số cạnh bị xóa là $\mathcal{O}(m)$. Do đó chi phí tìm luồng chặn trong một pha là

$$\mathcal{O}(n(n + n) + m) = \mathcal{O}(n^2 + m).$$

Trong cách đánh giá của bài gốc, lấy $\mathcal{O}(n^2)$ cho thành phần chính của việc tìm luồng chặn trên mỗi tầng. Kết hợp với nhiều nhất n pha, độ phức tạp tổng quát của MPM được ghi là

$$\mathcal{O}(n^3).$$

MPM có ý tưởng đẹp nhưng cài đặt rườm rà hơn Dinic. Theo cách nói hơi đùa của bản gốc, MPLA dễ hiểu và dễ viết cho dữ liệu vừa nhỏ; Dinic dễ hiểu, dễ viết và dùng tốt cho dữ liệu lớn hơn; còn MPM dễ hiểu nhưng viết phiền, nên trong thi đấu thông thường lợi ích thực tế của nó không rõ bằng Dinic.

7 Tổng kết

Ba thuật toán có thể nhìn nhanh như sau:

Thuật toán	Ý tưởng trong mỗi pha	Độ phức tạp	Đặc điểm
MPLA	nhiều BFS tìm đường tăng	$\mathcal{O}(nm^2)$	dễ hiểu, hợp dữ liệu vừa và nhỏ
Dinic	DFS tìm luồng chặn	$\mathcal{O}(mn^2)$	dễ cài, hợp dữ liệu lớn hơn
MPM	xử lý đỉnh có thông lượng nhỏ nhất	$\mathcal{O}(n^3)$	ý tưởng rõ, cài đặt phức tạp hơn

So sánh tổng thể, Dinic là lựa chọn rất đáng ưu tiên trong các kỳ thi tin học: nó dựa trên khái niệm phân tầng giống MPLA, nhưng tìm luồng chặn hiệu quả hơn nhiều; đồng thời cách cài đặt vẫn đủ ngắn và ổn định để dùng trong điều kiện thi. Với nhiều bài, nó có thể trở thành chìa khóa mở cửa lời giải, ngọn đèn chỉ đường tới kết quả đúng, đôi cánh giúp thuật toán bay qua giới hạn dữ liệu, và nguồn nước tưới cho bông hoa thành công, như lời văn đầy hình ảnh trong bản gốc.

Lời cảm ơn

Tác giả cảm ơn thầy Yu Xiaoqing đã cung cấp đề bài và dữ liệu; cảm ơn Li Tianyi, Hu Botao, Hu Gang, Shou Heming và huấn luyện viên Liu Rujia đã góp ý cho bản thảo hoặc chương trình.

Tài liệu tham khảo

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*.
2. M. H. Alsuwailayel, *Algorithms: Design Techniques and Analysis*.
3. Liu Rujia, Huang Liang, *The Art of Algorithms and Informatics Competitions*.

4. Wang Xiaodong, *Computer Algorithm Design Techniques and Analysis*.
5. Fred Buckley, Marty Lewinter, *A Friendly Introduction to Graph Theory*.
6. Zhou Yuan, solution report for the 2006 Anhui provincial informatics selection contest.

Phụ lục: đề bài mẫu

Lợi nhuận lớn nhất

Bài từ NOI 2006.

Mô tả bài toán. Công nghệ mới đang tác động mạnh tới thị trường truyền thông di động. Với các nhà khai thác, đây vừa là cơ hội vừa là thách thức. Công ty truyền thông CS&T thuộc tập đoàn THU cần chuẩn bị cho cuộc cạnh tranh của thế hệ công nghệ mới; chỉ riêng việc chọn vị trí trạm đã phải làm nghiên cứu thị trường, khảo sát địa điểm và tối ưu hóa.

Sau khảo sát, công ty có N vị trí có thể xây trạm trung chuyển tín hiệu. Do vị trí địa lý khác nhau, chi phí xây trạm tại mỗi nơi khác nhau; xây trạm i tốn P_i với $1 \leq i \leq N$. Ngoài ra có M nhóm người dùng kỳ vọng. Nhóm i được mô tả bởi A_i, B_i, C_i : họ sẽ dùng hai trạm A_i và B_i để liên lạc, đem lại lợi nhuận C_i cho công ty.

Công ty có thể chọn xây một số trạm và phục vụ một số nhóm người dùng. Hãy chọn sao cho lợi nhuận ròng lớn nhất:

$$\text{lợi nhuận ròng} = \text{tổng lợi nhuận phục vụ} - \text{tổng chi phí xây trạm}.$$

Dữ liệu vào. Dòng đầu có hai số nguyên dương N, M . Dòng thứ hai có N số nguyên $P_1, P_2, \dots, P_N$, là chi phí xây từng trạm. M dòng tiếp theo, dòng thứ i gồm ba số A_i, B_i, C_i mô tả nhóm người dùng thứ i .

Dữ liệu ra. In ra một số nguyên duy nhất: lợi nhuận ròng lớn nhất công ty có thể đạt được.

Ví dụ.

profit.in	profit.out
5 5	
1 2 3 4 5	
1 2 3	
2 3 4	4
1 3 3	
1 4 2	
4 5 3	

Giải thích ví dụ. Chọn xây các trạm 1, 2, 3, chi phí là 6, lợi nhuận thu được là 10, nên lợi nhuận ròng lớn nhất là 4.

Chấm điểm và ràng buộc. Bài không có chấm điểm từng phần: chương trình chỉ được điểm nếu kết quả trùng khớp hoàn toàn với đáp án. Với 80% dữ liệu, $N \leq 200, M \leq 1000$. Với 100% dữ liệu, $N \leq 5000, M \leq 50000, 0 \leq C_i \leq 100, 0 \leq P_i \leq 100$.

Trò chơi ma trận

Bài từ kỳ tuyển chọn đội tuyển Olympic Tin học thanh thiếu niên tỉnh Giang Tô JSOI 2006.

Mô tả bài toán. Với một ma trận 0-1, ta có thể thống kê số ô bằng 1 trên từng hàng và từng cột. Ví dụ, ma trận

1	0	0	0	1
0	1	1	1	3
1	0	1	0	2
2	1	2	1	

có tổng hàng lần lượt là 1, 3, 2, tổng cột lần lượt là 2, 1, 2, 1.

Cho một ma trận 0-1 cần dựng, biết số lượng 1 trên từng hàng là $r_1, r_2, \dots, r_n$, số lượng 1 trên từng cột là $c_1, c_2, \dots, c_m$. Đồng thời, một số ô được chỉ định bắt buộc bằng 0; bảo đảm mỗi hàng và mỗi cột có nhiều nhất một ô bị chỉ định như vậy. Hãy xác định có tồn tại ma trận 0-1 thỏa tất cả yêu cầu hay không.

Dữ liệu vào. Dòng đầu chứa số tự nhiên T , $1 \leq T \leq 10$, là số test. Mỗi test gồm: dòng đầu có hai số nguyên dương n, m , $1 \leq n, m \leq 1000$; dòng thứ hai có n số $r_1, \dots, r_n$ với $0 \leq r_i \leq m$; dòng thứ ba có m số $c_1, \dots, c_m$ với $0 \leq c_j \leq n$; dòng thứ tư có số tự nhiên t , là số ô bắt buộc bằng 0. Tiếp theo là t dòng, mỗi dòng gồm hai số là hàng và cột của một ô bị chỉ định. Hàng và cột được đánh số từ 1. Giữa hai test liên tiếp có một dòng trống.

Dữ liệu ra. Với mỗi test, in Yes nếu tồn tại ma trận thỏa yêu cầu, ngược lại in No.

Ví dụ.

matrix.in	matrix.out
2	
3 4	
1 3 2	
2 1 2 1	
2	
1 2	Yes
2 1	No
1 1	
1	
1	
1	
1 1	

Ràng buộc. Sáu test đầu thỏa $1 \leq n, m \leq 100$. Bốn test cuối thỏa $1 \leq n, m \leq 1000$.